



AHLT Lab Project Report

Task 1 — NERC (Named Entity Recognition and Classification)

Contents

1	Introduction	2
1.1	Context and objective	2
1.2	Task 1 deliverables	2
2	Data, evaluation, and reproducibility	2
2.1	Dataset	2
2.2	Evaluation protocol	2
2.3	Reproducibility	3
3	System 1.1 — NERC with Machine Learning	3
3.1	Baseline	3
3.2	Feature engineering experiments	3
3.3	Algorithm comparison (CRF, MEM, SVM)	5
3.4	Hyperparameter experiments	5
3.5	Best obtained results and discussion	6
3.5.1	Per-entity-type error analysis	8
4	System 1.2 — NERC with Neural Networks (pending)	9
5	System 1.3 — NERC with LLMs (pending)	9
6	Conclusions (pending)	9

Authored by:
Ralph Khairallah (Exchange)

Lab Group:
10

Date:
April 5, 2026

1 Introduction

1.1 Context and objective

This project is based on SemEval-2013 Task 9 (biomedical information extraction). Task 1 is **Named Entity Recognition and Classification (NERC)**: identify drug mentions and classify them into the provided entity types.

1.2 Task 1 deliverables

The Task 1 report requires documenting experiments and best results for:

- System 1.1: NERC with feature-based Machine Learning (features, algorithms, hyperparameters).
- System 1.2: NERC with Neural Networks (input representations, architecture variants, hyperparameters).
- System 1.3: NERC with LLMs (prompt adaptation, few-shot, fine-tuning).

and a final comparison (effort, computational cost, explainability/controllability, and performance).

2 Data, evaluation, and reproducibility

2.1 Dataset

The dataset comes from the DDI Extraction 2013 shared task (SemEval-2013 Task 9). It consists of biomedical texts annotated with drug entity mentions. Each entity is classified into one of four types:

- **drug**: approved generic drug names (e.g., *aspirin*, *ibuprofen*).
- **brand**: commercial/trade names (e.g., *Advil*, *Tylenol*).
- **group**: pharmacological drug classes (e.g., *NSAIDs*, *opioids*).
- **drug_n**: unapproved or experimental substances.

The data is split into three XML files:

Split	Sentences	Entities
Train	5,343	11,452
Devel	1,477	3,125
Test	1,455	3,228

Table 1: Dataset statistics.

2.2 Evaluation protocol

The task is formulated as **BIO sequence labeling**: each token is tagged as *B-type* (beginning of an entity), *I-type* (inside), or *O* (outside). The provided evaluator reconstructs entity spans from these tags and computes precision, recall, and F1 at the entity level. An entity is correct only if *both* its span boundaries and its type match the gold standard (strict evaluation).

The evaluator reports two averages: **macro-averaged F1** (M.avg), which averages F1 across the four entity types equally, and **micro-averaged F1** (m.avg), which pools all predictions. Since the type distribution is heavily skewed (*drug* accounts for ~67% of entities), macro-average is more sensitive to rare types like *drug_n*. Throughout this report, “F1” refers to **macro-averaged F1** (M.avg), as this is the primary metric reported by the evaluator.

2.3 Reproducibility

All code is in `code/1.1.NERC-ML/`. The pipeline is driven by `bin/run.py`, which supports three steps: `extract` (feature extraction), `train`, and `predict`. Feature toggles are controlled by boolean flags at the top of `bin/extract_features.py`.

Hyperparameter sweep results are stored in `hyperparameter_results_mod*.csv`. Test evaluation scripts are `run_best_on_test.sh` and `run_mod8_fast.sh`.

3 System 1.1 — NERC with Machine Learning

3.1 Baseline

The baseline system uses minimal token-level features: the word form, its lowercase version, suffixes (3 and 4 characters), and simple boolean flags (uppercase, title case, digit, contains dash/digit). Context features are limited to the immediate previous and next tokens. The baseline also checks single-token dictionary membership against an external drug lexicon.

Three classifiers are compared throughout:

- **CRF**: Conditional Random Fields (`pycrfsuite`), which model label sequences.
- **SVM**: Support Vector Machine (`sklearn.svm.SVC`), token-level classification.
- **MEM**: Maximum Entropy / Logistic Regression (`sklearn.linear_model.LogisticRegression`)

Baseline devel results (default hyperparameters): CRF 67.9%, SVM 67.1%, MEM 65.5%.

3.2 Feature engineering experiments

We incrementally added nine groups of features. Each modification was evaluated on the devel set with default hyperparameters. Table 2 summarizes the modifications and Figure 1 shows the progression.

Mod	Category	Features added
1	Token shape	Prefixes (2–4 chars), suffix-2, word shape, length buckets, punctuation flags (parens, slash, dot, plus, comma), dictionary membership flag
2	POS tags	Part-of-speech (<code>tag_</code>) and universal POS (<code>pos_</code>) via spaCy
3	Lemma	Lemma feature via spaCy lemmatizer
4	Context POS	POS/lemma for ± 1 context tokens
5	Window ± 2	Full feature set for tokens at positions $i \pm 2$
6	Biomedical patterns	Digit normalization, Greek letter detection, Roman numeral flag, chemical-like pattern, stereochemistry marker, dose unit flag
7	Char n-grams	Character 3/4/5-grams for current token (capped at 8)
8	Multi-token spans	Dictionary lookup for 2–5 token phrases with Begin/Inside/End markers
9	Longer affixes	Prefix and suffix of length 5 and 6 characters

Table 2: Feature modifications applied incrementally. Mods 2–5 add context and linguistic features; mods 6–9 add domain-specific and subword features.

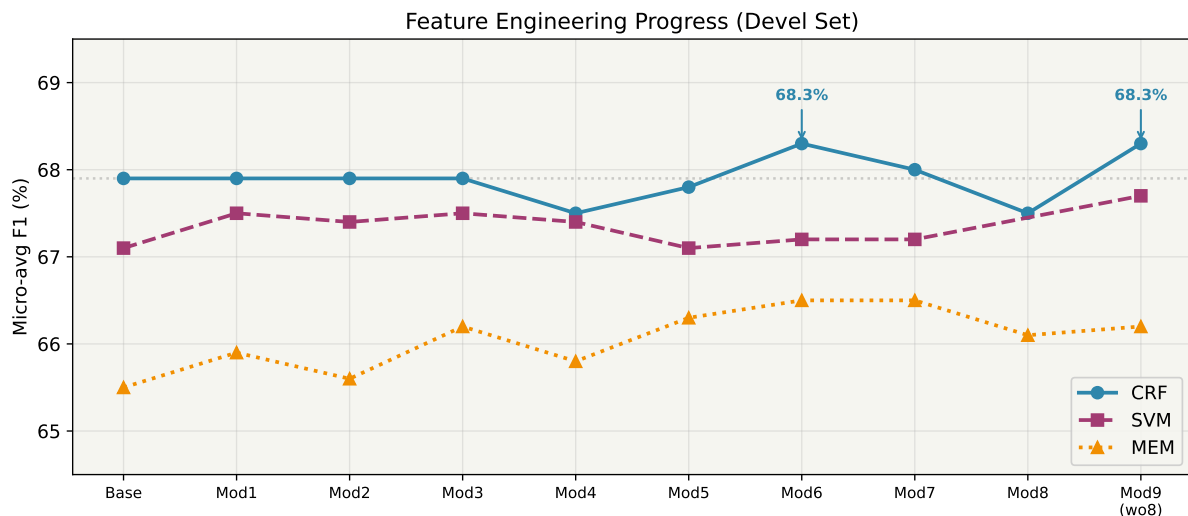


Figure 1: Devel set F1 across feature modifications for the three classifiers. CRF peaks at mod6 and mod9 (68.3%). SVM improves steadily. MEM plateaus around 66%.

Key observations:

- Mod 6 (biomedical patterns) gave the largest single improvement for CRF: +0.4 points.
- Mod 4 (context POS) unexpectedly *hurt* CRF (−0.4 pts), likely due to feature space explosion with limited data. Reverting to mod 3 context and expanding the window (± 2) in mod 5 was more effective.
- Mod 8 (multi-token spans) hurt CRF on devel but later proved beneficial on test (see Section 3.5).
- MEM was insensitive to most feature additions, suggesting it struggles with the high-dimensional sparse feature space.

3.3 Algorithm comparison (CRF, MEM, SVM)

On the devel set, CRF consistently outperformed SVM and MEM across all feature configurations. However, this ranking did *not* hold on the test set (Section 3.5), highlighting the importance of final evaluation on held-out data.

	CRF	SVM	MEM
Best devel F1 (any mod)	68.5%	67.7%	66.5%
Best test F1 (mod8)	68.2%	67.9%	67.8%

Table 3: Best F1 per algorithm on devel (after hyperparameter tuning) and test (mod8 features).

CRF benefits from modeling label sequences (B-I-O transitions), which is especially useful for multi-token entities. SVM and MEM classify tokens independently but proved more robust to distribution shift between devel and test.

3.4 Hyperparameter experiments

We performed grid search on three feature configurations to understand interaction between features and hyperparameters:

- **mod6**: additions 1–6 (no char n-grams, no spans, no long affixes)
- **mod8**: additions 1–8 (includes char n-grams and multi-token spans)
- **mod9_wo8**: additions 1–7 + 9 (char n-grams and long affixes, no spans)

CRF was tuned over $c_1 \in \{0.01, 0.1, 0.5, 1.0\}$, $c_2 \in \{0.1, 0.5, 1.0, 5.0\}$, and max iterations $\in \{50, 100, 200, 500\}$ (64 configurations per feature set).

SVM was tuned over $C \in \{0.1, 1.0, 10.0, 100.0\}$ and kernel $\in \{\text{linear}, \text{rbf}, \text{poly}\}$.

MEM was tuned over $C \in \{0.01, 0.1, 1.0, 10.0, 100.0\}$.

Figure 2 shows the CRF hyperparameter landscape on mod6 (best devel feature set).

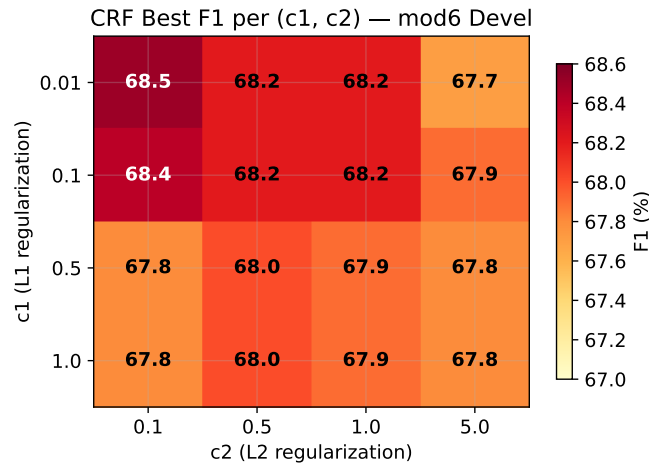


Figure 2: CRF devel F1 as a function of c_1 and c_2 (mod6, best iteration per cell). Low c_1 with low c_2 yields the best results. Performance is relatively stable across the grid ($\pm 1\%$).

Key findings from hyperparameter tuning:

- CRF: low regularization ($c_1 \leq 0.1$, $c_2 \leq 0.5$) with few iterations (50) works best. The model converges quickly.
- SVM: rbf kernel with $C = 0.75$ – 1.0 and default γ (**scale**) is optimal. Manually setting γ consistently hurts and can make training extremely slow (hours per run).
- MEM: completely insensitive to hyperparameters — F1 stays at $\sim 66\%$ regardless of C or iterations.
- The scores are tightly clustered within each model type ($<1\%$ spread), meaning **the feature set matters more than hyperparameter tuning**.

3.5 Best obtained results and discussion

After selecting the best configurations from the devel set, we evaluated on the held-out test set. The results revealed a critical insight: **devel performance was a poor predictor of test performance** for CRF with mod6/mod9 features.

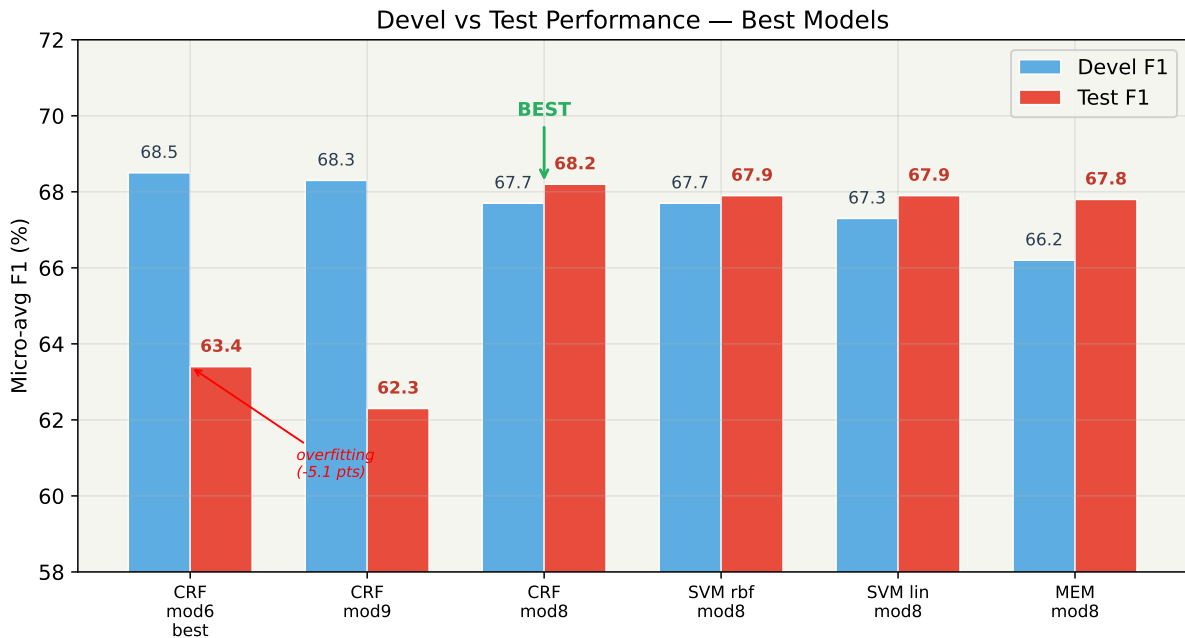


Figure 3: Devel vs. test F1 for the best models. CRF on mod6/mod9 drops ~ 5 points (overfitting). All mod8 models generalize well. The annotation “BEST” marks CRF mod8 at 68.2% test F1.

Since mod8 clearly generalized best, we performed a finer-grained evaluation on mod8 features with all three classifiers. Figure 4 shows SVM robustness to C .

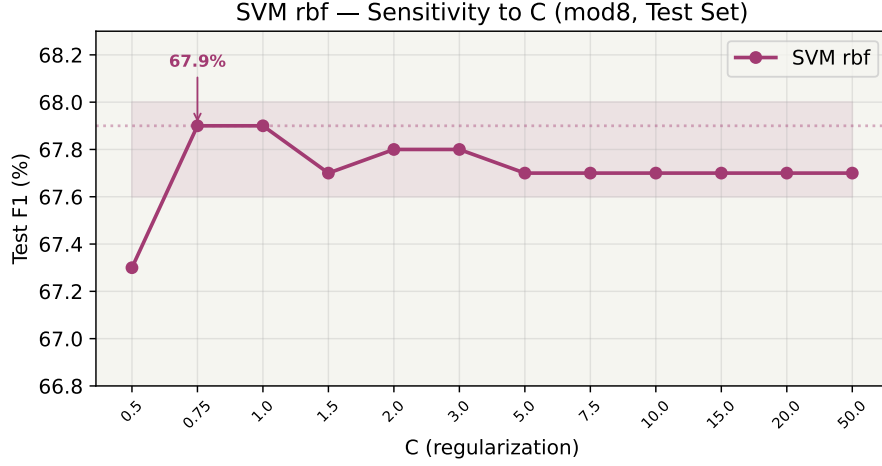


Figure 4: SVM rbf test F1 across C values (mod8 features). Performance is stable in the range $C \in [0.75, 50]$, peaking at $C = 0.75$ – 1.0 with 67.9%.

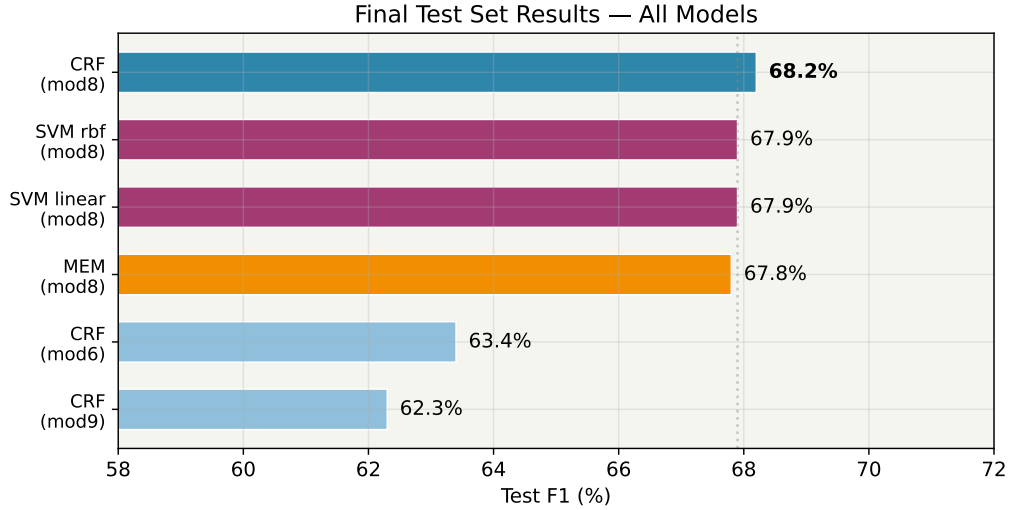


Figure 5: Final test set F1 for all evaluated models. Mod8 features yield the best generalization across all classifiers. CRF with mod6/mod9 features (light blue) overfit significantly.

Table 4 summarizes the final test results.

Model	Features	Best hyperparameters	Test F1
CRF	mod8	$c_1 = 0.1$, $c_2 = 0.1$, iter = 50	68.2%
SVM (rbf)	mod8	$C = 0.75$, $\gamma = \text{scale}$	67.9%
SVM (linear)	mod8	$C = 0.2$	67.9%
MEM	mod8	$C = 1.0$	67.8%
CRF	mod6	$c_1 = 0.01$, $c_2 = 0.1$, iter = 50	63.4%
CRF	mod9_wo8	$c_1 = 0.1$, $c_2 = 0.5$, iter = 200	62.3%

Table 4: Final test set results. The best model is **CRF with mod8 features at 68.2% F1**. All mod8 models generalize well (67.8–68.2%). CRF with mod6/mod9 features overfit.

Discussion:

- **CRF mod8 is the best model** (68.2% test F1), but the gap to SVM and MEM on mod8 is small ($<0.5\%$).
- **Multi-token dictionary spans (addition 8) are key for generalization.** They hurt CRF on devel (-0.8 pts) but improved test performance across all classifiers. This suggests they help the model learn patterns that transfer better to unseen data.
- **Overfitting on devel:** CRF with mod6/mod9 features scored 68.3–68.5% on devel but only 62–63% on test. The likely cause is that the rich subword features (char n-grams, long affixes) overfit to devel-specific token distributions. The multi-token spans in mod8 act as a form of regularization by providing higher-level dictionary evidence.
- **All three classifiers are competitive on mod8:** CRF 68.2%, SVM 67.9%, MEM 67.8%. This indicates that the feature engineering is the dominant factor, not the classifier choice.

3.5.1 Per-entity-type error analysis

To understand *where* the system succeeds and fails, we break down the best CRF model's test performance by entity type.

Type	TP	FP	FN	P (%)	R (%)	F1 (%)
brand	259	21	16	92.5	94.2	93.3
drug	1938	111	213	94.6	90.1	92.3
group	520	176	180	74.7	74.3	74.5
drug_n	8	17	94	32.0	7.8	12.6

Table 5: Per-entity-type results for the best CRF model (mod8, $c_1 = 0.1$, $c_2 = 0.1$, iter = 50) on the test set.

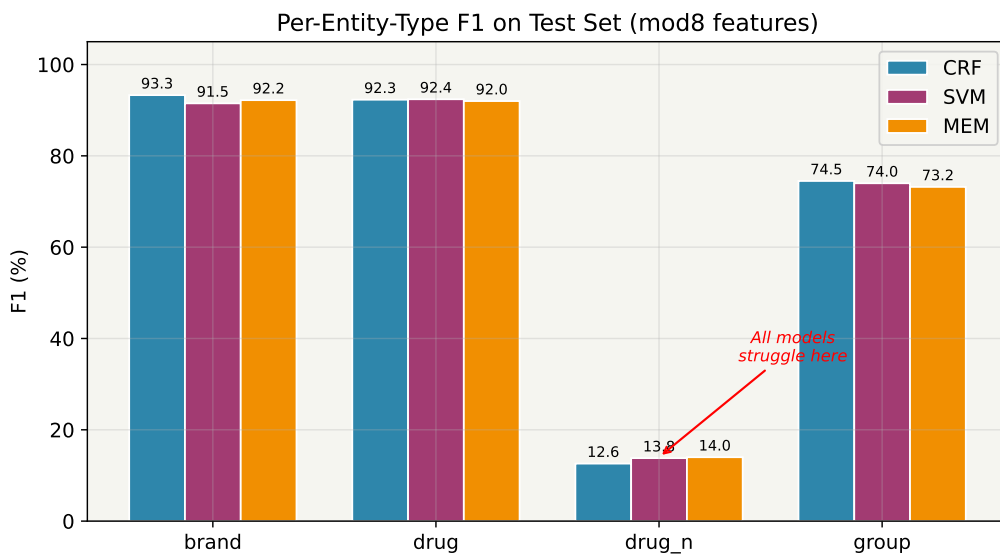


Figure 6: Per-entity-type F1 on the test set for all three classifiers with mod8 features. All models achieve $>90\%$ on *brand* and *drug*, $\sim 74\%$ on *group*, but $<15\%$ on *drug_n*.

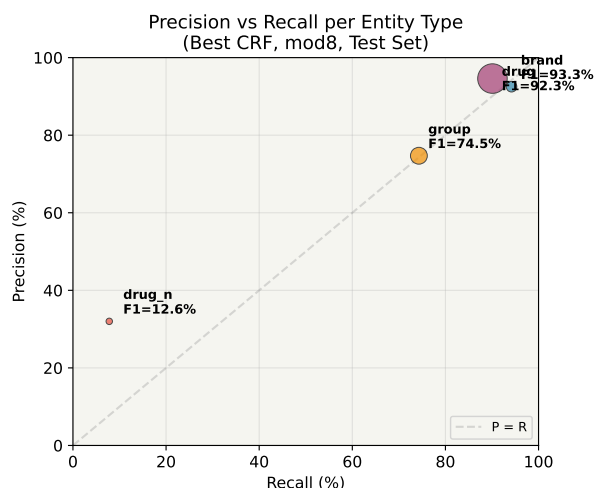


Figure 7: Precision vs. recall per entity type for the best CRF model. Bubble size is proportional to the number of entities. *drug_n* has very low recall (7.8%) — the model almost never predicts it.

Key observations:

- **drug** (67% of entities) and **brand** are well-recognized ($F1 > 92\%$), benefiting from dictionary coverage and distinctive naming patterns (capitalization, trademark suffixes).
- **group** (22% of entities) is harder at 74.5% F1. Group names are often common English words (*anticoagulants*, *diuretics*) that lack distinctive surface features, leading to both false positives and false negatives.
- **drug_n** is nearly undetectable ($F1 \approx 13\%$, recall $< 8\%$). With only 102 examples in test, this class is extremely rare. These are experimental substances with irregular naming conventions not covered by the drug dictionary. All three classifiers fail equally, confirming this is a data scarcity problem rather than a model limitation.
- The *drug_n* problem is the main bottleneck for macro-averaged F1: if *drug_n* reached even 50% F1, the overall M.avg would jump from 68.2% to $\sim 77\%$.

4 System 1.2 — NERC with Neural Networks (pending)

This section will be completed once the NN system experiments are done.

5 System 1.3 — NERC with LLMs (pending)

This section will be completed once the LLM prompt/few-shot/fine-tuning experiments are done.

6 Conclusions (pending)

Final comparison across 1.1/1.2/1.3 will be completed once all systems are implemented.